MANCHESTER 1824
The University of Manchester

MATH49111/69111: Scientific Computing

Variables

Matthias Heil

M.Heil@maths.manchester.ac.uk

A more detailed look at C++ syntax

```cpp
#include <iostream>

int main()
{
  double sum = 0.0;        // Set up variables
  int n_terms = 100, i=1; // and initialise them

  while (i<=n_terms) // Loop and sum terms
  {
    sum += 1.0/(i*i);
    i++;
  }

  std::cout << "Sum of sequence after " << n_terms
            << " terms = " << sum << std::endl; // output result

  return 0;
}
```

Main function

```
#include <iostream>

int main()
{
  double sum = 0.0;  // Set up variables
  int n_terms = 100, i=1;

  while (i<=n_terms) /* Loop and sum terms */
   {
     sum += 1.0/(i*i);
     i++;
   }

  std::cout << "Sum of sequence after " << n_terms
            << " terms = " << sum << std::endl; // output result
  return 0;
}
```

Main function

```
int main()
{
  // Your code here

  return 0;
}
```

- The `main` function is called by the operating system when the program starts (it defines the *entry point*).
- All C++ programs must define a `main` function
- For now take the syntax as read; we will look in more detail later at the syntax for functions

Variables

```
#include <iostream>

int main()
{
    double sum = 0.0;  // Set up variables
    int n_terms = 100, i=1;

    while (i<=n_terms) /* Loop and sum terms */
    {
        sum += 1.0/(i*i);
        i++;
    }

    std::cout << "Sum of sequence after " << n_terms
         << " terms = " << sum << std::endl; // output result

    return 0;
}
```

Variables

```
double sum = 0.0;
int n_terms = 100;
```

- Variables contain data (often numerical), which is modified by the program
- Variables have a *type*, which defines the kind of data stored
  - C++ defines a number of fundamental data types, for integers, fractional values etc. (we have already seen `int` and `double`)
  - The C++ standard library defines many more
  - New types can be defined in a C++ program
  - The type of a variable cannot be changed at run-time
- In C++ we must declare a variable (and its type) before using it.
- Here we are declaring the two variables and *initialising* them using the *literal values* `0.0` and `100`.

Variables — built-in data types

Type	*Bytes*	*Description*	*Literals*	*Notes*
`char`	1	A text character	`'A'`	Single quotes
`int`	4	An integer	`5` `-20`	Max. and min. values of at least $\approx \pm 2 \times 10^9$
`unsigned`	4	A natural number	`6`	Range is 0 to at least $\approx 4 \times 10^9$

`double`	8	Fractional number	`3.142`
			`-5.`
			`3.5e8`
				Floating point, $\approx$ 16 digits of accuracy, range of between $\pm(10^{-308}$ to $10^{308})$

`float`	4	Fractional number	`3.142f`	Floating point, $\approx$ 7 digits of accuracy
`bool`	1	Boolean value	`true` `false`	`true` $= 1$, `false` $= 0$

…plus various other sizes of integer, and the null `void` type

Variables — a stylistic interlude (i)

- Like everything in code, the type of a variable conveys information: if you use an `int` you're implying that the variable can meaningfully take negative values. So if you use an `int` as a counter you're suggesting that negative values can occur (and are possibly used to indicate some special cases). If that's not intended, use an `unsigned` instead.
- Similarly, when dealing with `bools` avoid using the fact that the compiler happily treats them as a tiny subset of `ints` via `true` $= 1$, `false` $= 0$.
- Names of variables are important and should explain their intended usage.
- Even the way you typeset a variable name (`lowercase`, `UPPERCASE`, `Capitalised`, `CompoundNameWithUpperCase`, `compound_name_with_underscore`, ...; remember that C++ is case sensitive), can be used to convey information.
- Adopt a consistent style!

Variables — a stylistic interlude (ii)

- As of the C++11 standard, the type of variables can be inferred automatically, using the `auto` type.
- So these are (sort of) equivalent

```
int i = 0;
double x = 1.3;
```

```
auto i = 0;
auto y = 1.3;
```

- Note that the `auto` declaration must be accompanied by an initialisation – it is what (sort of) determines the type of the variable. The following is not allowed:

```
auto i;
i=0;
```

- Unlike in python, once the type of a variable is set you can't change it (and thank goodness for that!)

Variables — a stylistic interlude (iii)

- Please don't use the `auto` statement at all!

```
// C++ will make this an int. Or will it? Could it be an unsigned?
auto i = 0;

// C++ will make this a double. Or will it? Could be a float?
auto y = 1.3;
```

- There are, of course, rules how the automatic type assignment is done but the use of the `auto` statement removes clarity from the code.
- Towards the end of this course I will show you a few exceptions where the use of `auto` helps deal with very complicated variable types, but even there it's best used with caution.

Variables — a stylistic interlude (iv)

Always initialise variables!

- This is legal code:

```
// Declare variable
int i;

// Give it a value
i=1;
```

- ...and so is this:

```
// Declare variable
int i;

[... lots of code ...]

// What's the value of i?
int j=i*i;
```

- You may think that the compiler initialises variables to "something sensible".
- However, for many data types it's not clear what "something sensible" (usually the equivalent of zero) is.
- Initialisation takes time so the helpful (?) automatic initialisation will be suppressed if you compile the code with full optimisation. In this case the value of the uninitialised variables is undefined, resulting in extremely hard to debug code!

Variables — type conversion

From above:

> *"Unlike in python, once the type of a variable is set you can't change it (and thank goodness for that!)"*

- ...but you make a new variable of a different type:

```
unsigned i=4;

// The rhs turns i into a double; we now have x=4.0.
// This is old-skool C syntax
double x = (double)(i);

// Alternative C++ syntax: calls a ``constructor'' for a double
double y = double(i);

// Alternative C++ syntax: recasts int to a double
double z = static_cast<double>(i);
```

Variables — type conversion: more stylistic edicts

- **Note:** C++ will do type conversions automatically, but it shouldn't and you shouldn't rely on it!
- So please don't do this:

```
unsigned i=4;

// Yes, this compiles, but lacks clarity
double x = i;
```

Variables — scope

- We may declare variables anywhere in a function before we use them (here at the top of the function).
- The variables are declared inside the `main` function, so they can only be accessed inside this function (this is their *scope*).
- More generally, variables will be localised to the *block* (delimited by braces `{}`) in which they are declared.
- What are the values of `i`, `j` and `k` after each line?

```
int i = 0; // ?
int j = 1; // ?
{
    int i = 2; // ?
    int k = 3; // ?
}
// ?
```

Variables — scope

- We may declare variables anywhere in a function before we use them (here at the top of the function).
- The variables are declared inside the `main` function, so they can only be accessed inside this function (this is their *scope*).
- More generally, variables will be localised to the *block* (delimited by braces `{}`) in which they are declared.
- What are the values of `i`, `j` and `k` after each line?

```
int i = 0; // i=0; j, k undefined
int j = 1; // i=0; j=1; k undefined
{
    int i = 2; // i=2; j=1; k undefined
    int k = 3; // i=2; j=1; k=3
}
// i=0; j=1; k undefined
```

Variables — scope

Note the difference:

- What are the values of `i`, `j` and `k` after each line?

```
int i = 0; // i=0; j, k undefined
int j = 1; // i=0; j=1; k undefined
{
 int i = 2; // i=2; j=1; k undefined
 int k = 3; // i=2; j=1; k=3
}
// i=0; j=1; k undefined
```

```
int i = 0; // i=0; j, k undefined
int j = 1; // i=0; j=1; k undefined
{
 i = 2; // i=2; j=1; k undefined
 int k = 3; // i=2; j=1; k=3
}
// i=2; j=1; k undefined
```

- Variables outside the scope are “visible” inside, unless they are re-defined inside.

Examples

```
// Declare and initialise three ints
int i = 4;
int j = 3;
int k = 2;
```

```
int i = 4, j = 3, k = 2; // multiple declarations (of one type) in one statement
```

```
int i = 4, j = 3, k = 2; // different types require a
double a = 4.0, b = 5.0; // new statement
```

```
int i, j, k = 2; // initial values recommended but not required
```

```
int i{4}, j{3}, k{2}; // equivalent to i = 4, etc.
```